

ELECTRONIC HOBBYISTS CORNER

[Home](#)
[Site Information](#)
[Contact Details](#)

[Projects](#)
[Technical Articles](#)
[Reference Section](#)
[Component Tutorials](#)

[Site Map](#)
[Links](#)

An Introduction To Microcontrollers

[Definition of a Microcontroller](#)
[Why use a Microcontroller?](#)
[What a Microcontroller must have](#)
[Real Microcontrollers](#)
[The BASIC Stamp](#)
[The PICmicro](#)
[BASIC Stamp Software](#)
[PIC Software](#)
[Conclusion](#)

Definition of a Microcontroller:

First up, just so we're all clear on at least one thing, what is a microcontroller? Technical definitions can be as vague as "Computer Chips" and personal opinion is bound to vary. I like to think of microcontrollers as programmable logic; integrated circuits which can be programmed to do a number of (hopefully logical) things with their pins.

The applications for micros are endless. It's possible to control just about anything electronic with software written by the user and downloaded/programmed into the microcontroller.

Why use a Microcontroller:

Two words; miniaturization, flexibility. Some of the more powerful microcontrollers can easily replace a few hundred logic IC's and achieve a more reliable result. Micros are best suited to situations where some degree of intelligence is required. (ie. decision making) They are by no means restricted to this.

I'm sure everyone will agree, it's easier to edit a text file than rewire a PCB to fix an error. Life can be made a lot easier by dictating the operation of a circuit with software, rather than component layout.

What a Microcontroller must have:

Most general purpose microcontrollers have the following:

- A CPU to execute programmed code.
- Some memory, both RAM and ROM.
- Internal timers.
- An Input/Output system, usually in the form of I/O pins.

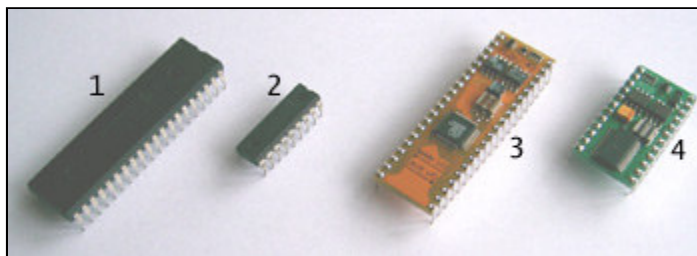
Some micros will have extras:

- ADCs and DACs for playing games in the non-digital world.
- UARTs for serial communications.
- Internal EEPROM for storing long term data.
- PWM modules for analog output.

Because microcontrollers execute software they are essentially digital devices. This doesn't mean that they can't work with analog electronics, but it does mean that any work with analog electronics is done in a digital form.

Real Microcontrollers:

Now I'll introduce some real world examples.



1. *PIC18F452*: One of the more powerful PICmicro's.

2. *PIC16F628*: Mid range PICmicro. Less I/O pins, less program space... etc.

3. *BASIC Stamp 2p40*: The most advanced of the BASIC Stamp series.

4. *BASIC Stamp 2*: Joe Bloggs BASIC Stamp, actually a PICmicro at heart.

Another microcontroller I'm told can be quite useful is the PICAXE. My understanding is that these devices are actually PICmicro's with an interpreter firmware which executes BASIC tokens, (just like the BASIC Stamp) the difference being that they store the user code in the chip itself rather than an external EEPROM. The result is a single chip (rather than a small circuit board) and considerably reduced prices. The cheapest being under NZ\$10. PICAXE micros and starter kits can be purchased from the [South Island Component Centre](#). The PICAXE homepage can be found [here](#).

Many, many, many, many more different microcontrollers exist. The PICmicro family has several hundred members alone, only two of which are above. Because I only have experience with the BASIC

Stamp and PICmicro I won't be going into detail on other systems, but a lot of the information in this article will apply to most micros.

The BASIC Stamp:

The BASIC Stamp is a good entry point, in my opinion. They are as fast as the average hobbyist needs for most applications, are extremely easy to write software for and require only a serial cable for programming. One of the biggest advantages of using a BASIC Stamp is that all the functions (PWM, serial I/O... etc) can be performed on any I/O pin, rather than the pin dedicated for that function.

As you might have guessed from the name, BASIC Stamps are programmed in yet another variation of BASIC, (Beginners All-Purpose Symbolic Instruction Code) called Parallax BASIC or just PBASIC. It's very easy to learn and quite powerful with some of the more advanced BASIC Stamp modules.

The BASIC Stamp is somewhat different to most microcontrollers in construction. Rather than being a self-contained chip they are constructed on a small PCB which usually fits into a standard IC footprint. They use another microcontroller to run a PBASIC interpreter which fetches "tokens" out of an external EEPROM and executes them. This EEPROM can also hold user data, in addition to the internal RAM.

The most annoying thing about the BASIC Stamp is the cost. The lower end modules are in the range of NZ\$100 to NZ\$150, and things like the BS2p40 are close to NZ\$300. Not exactly the in the hobbyists budget, but it doesn't stop most people. As there's only a few different modules, I'll go over each on briefly.

<i>BASIC Stamp 1</i>	
The BS1 was the first Stamp. It's slow, has very little code space and has less I/O pins than most microcontrollers but can still do some remarkable things. Parallax have recently revived this relic by making it compatible with their new Windows IDE.	
Interpreter:	PIC16C56
Clock Speed:	4MHz
Execution Rate:	~2,000 Tokens/sec
User RAM:	16 Bytes
Code Space:	256 Bytes (~80 Tokens)
I/O Pins:	8
Programming Interface:	COM or LPT ports.

BASIC Stamp 2

The BS2 is a good starting point when getting into BASIC Stamps. Considerably faster than the BS1 with a more flexible instruction set and more space for code.

Interpreter:	PIC16C57
Clock Speed:	20MHz
Execution Rate:	~4,000 Tokens/sec
User RAM:	32 Bytes
Code Space:	2,048 Bytes (~500 Tokens)
I/O Pins:	16 (two dedicated RS-232)
Programming Interface:	COM port.

BASIC Stamp 2e

The BS2e is the BS2sx's (below) younger sibling. It runs at a speed more like the BS2, but has the features of the BS2sx like program slots and scratch-pad RAM.

Interpreter:	SX28
Clock Speed:	20MHz
Execution Rate:	~4,000 Tokens/sec
User RAM:	32 Bytes
Scratch-Pad RAM:	64 Bytes
Code Space:	8 x 2,048 Bytes (~4,000 Tokens)
I/O Pins:	16 (two dedicated RS-232)
Programming Interface:	COM port.

BASIC Stamp 2sx

Prior to the BS2p modules, this was the king. Features are almost identical to the BS2e but it runs twice as fast, and needs 3 times as much supply current.

Interpreter:	SX28
Clock Speed:	50MHz
Execution Rate:	~10,000 Tokens/sec
User RAM:	32 Bytes
Scratch-Pad RAM:	64 Bytes
Code Space:	8 x 2,048 Bytes (~4,000 Tokens)
I/O Pins:	16 (two dedicated RS-232)
Programming Interface:	COM port.

BASIC Stamp 2p

Comes in two flavours, 24pin and 40pin. (16 and 32 I/O pins respectively) The BS2p is faster than all the rest and is somewhat more efficient than the BS2sx. The BS2p40 is my choice, price aside....

Interpreter:	SX48
Clock Speed:	20MHz
Execution Rate:	~12,000 Tokens/sec
User RAM:	38 Bytes
Scratch-Pad RAM:	128 Bytes
Code Space:	8 x 2,048 Bytes (~4,000 Tokens)
I/O Pins:	16 or 32 (two dedicated RS-232)
Programming Interface:	COM port.

BASIC Stamp 2pe

Slower than the BS2p but with all the features, plus twice the EEPROM space. (only half can be used for code) The BS2pe is optimized for low power consumption when used correctly.

Interpreter:	SX48
Clock Speed:	8MHz
Execution Rate:	~6,000 Tokens/sec
User RAM:	38 Bytes
Scratch-Pad RAM:	128 Bytes
Code Space:	16 x 2,048 Bytes (8 x 2,048 for code)
I/O Pins:	16 (two dedicated RS-232)
Programming Interface:	COM port.

Javelin Stamp

I've never used a Javelin Stamp, but I know this much; It's programmed in a Java variant which is object oriented, making it considerably more powerful than PBASIC. It also has 32KB of continuous memory, rather than program slots.

Interpreter:	SX48
Clock Speed:	25MHz
Execution Rate:	~8,500 Tokens/sec
User RAM:	32,768 Bytes
Code Space:	32,768
I/O Pins:	16
Programming Interface:	COM port.

Overall, the BASIC Stamp is worth looking at, especially if you are just starting with microcontrollers and don't know a lot about assembly language programming.

The PICmicro:

The PIC differs from the BASIC Stamp in many, many ways. Firstly, they are completely self-contained processors with the CPU, memory and I/O all in a single package. Some of the more recent PIC's can run on an internal RC oscillator and therefore require no external resonator or crystal.

Being a RISC (Reduced Instruction Set Computer) processor, the instructions executed by the PIC do very little by themselves meaning simple arithmetic and I/O operations require the execution of several instructions. Thus using a PIC is a little harder than a BASIC Stamp. You can write your code in higher level languages like C and BASIC, but you sacrifice the speed and efficiency of assembly language programming. Compilers for higher level languages translate each statement into multiple instructions, some of which are not always needed but cannot be omitted.

Unlike the BASIC Stamp, PIC's support interrupts. An interrupt is triggered by an event such as a timer overflowing, an I/O pin changing state or an analog to digital conversion completing. When an interrupt occurs the micro automatically executes code (written by the user) to "service" the interrupt, which involves finding out what caused the interrupt and taking appropriate action. Interrupts allow the micro to perform other tasks while still being able to react to certain events without having to poll for those events. (Polling involves constantly checking whether the event has occurred or not)

Getting your code into a PIC is more difficult than for a BASIC Stamp. It must be programmed by an external device called a "programmer" (how imaginative) which interfaces to a PC through either a serial or parallel port. There's two ways to do this, high voltage (HVP) and low voltage programming (LVP).

High voltage programming requires the programmer to apply 13 volts to the PICmicro's /MCLR (reset) pin in order to enable the PIC to receive the program. As the PIC itself runs on a 5 volt supply the use of HVP programming ensures the micro cannot falsely program itself during normal operation. Low voltage programming provides a means of programming a PIC with 5 volts, but requires a dedicated I/O pin to initiate programming mode. (by driving it high) This I/O pin cannot be used by the user as normal operation could easily corrupt the devices programming.

Many DIY programmers are available for free over the internet and can be built for as little as NZ\$20. Almost all of these will be

"prototype" programmers, meaning they simply write and verify the program code. "Production" programmers must write and verify the code over the entire tolerable Vcc range, usually 4.5-5.5 volts. This is not necessary for most hobbyists. I strongly recommend the use of a HVP programmer, as LVP programming can only be enabled by a HVP programmer. (not to mention that less PIC's support LVP) I would also avoid programmers which derive their power (and 13V signal) from the RS-232 line, as some modern COM port do not provide significant voltage swings on their COM ports. If you want to use a laptop to write your software this is even more important.

Of the many designs out there, I would recommend the [Wisp628](#) programmer from [Technische Informatica](#). It uses a charge pump to get 13.5 volts (for HVP) from a 5 volt supply allowing it to be powered by the target circuit, and uses a MAX232 line driver to ensure compatibility with all computers.

I have recently posted some details on my own implementation of the Wisp628 PIC Programmer and the upcoming Wisp628 IDE software. [Click here](#) to check it out.

Below is a brief summary of a few micros I have used, the PIC16F628 and PIC18F452.

PIC16F84	
The PIC18C84 (EEPROM) and PIC18F84 (Flash) used to be the only re-programmable PIC's available and were therefore <i>the</i> hobbyists PIC's. Now the range of re-programmable micros is quite extensive and the '84 has been made obsolete, but many a project found online will still contain one. (in most cases you can use a PIC16F628 instead, they're cheaper)	
Package:	18pin Plastic DIP (also available in a few SMD packages)
Instructions:	35 (14bits wide)
Maximum Speed:	20MHz Clock / 5MHz Instruction Cycle (200ns)
Code Space:	1,048 x 14bit Words
Data (RAM) Space:	68 Bytes
EEPROM Space:	64 Bytes
I/O Pins:	13
Features:	8bit Timer

PIC16F628
This PIC is a recent replacement for the original hobbyists PIC, the PIC18F84, which is now obsolete. The '628 is designed to be mostly backwards compatible with the '84 and '84A but includes a few extra

features like a USART module and two more timers.

Package:	18pin Plastic DIP (also available in a few SMD packages)
Instructions:	35 (14bits wide)
Maximum Speed:	20MHz Clock / 5MHz Instruction Cycle (200ns)
Code Space:	2,048 x 14bit Words
Data (RAM) Space:	224 Bytes
EEPROM Space:	128 Bytes
I/O Pins:	16 (2 8bit ports)
Features:	USART, ADC, PWM, 3 Timers

PIC18F452

The "enhanced" (18F & 18C) PIC's are the fastest of the entire family, and the PIC18F452 is at the top end of the 18F range with 33 I/O pins, 32KB of program space and an extensive instruction set.

Package:	40pin Plastic DIP (also available in a few SMD packages)
Instructions:	71 (16bits wide)
Maximum Speed:	40MHz (10MHz + PLL) Clock / 10MHz Instruction Cycle (100ns)
Code Space:	16,384 x 16bit Words
Data (RAM) Space:	1,536 Bytes
EEPROM Space:	256 Bytes
I/O Pins:	33 (3 8bit ports, 1 6bit port, 1 3bit port)
Features:	USART, MSSP, ADC, PWM, 4 Timers, Oscillator Switching, Parallel Slave Port

BASIC Stamp Software:

Now that we're vaguely familiar with my two favourite microcontrollers I'll shed some light on writing programs for them. This is not a full blown tutorial, just a few bits of information to give an impression of what's required.

As mentioned earlier the BASIC Stamp is programmed in a variant of BASIC called PBASIC. Most PBASIC statements have the layout shown below.

```
STATEMENT Pin, Data, Variable
```

STATEMENT is the instruction name, like HIGH or LOW which set the voltage state on any I/O pin. *Pin* tells the BASIC Stamp what I/O pin to perform the instruction on, (this can usually be any I/O pin) *Data* is any number of comma delimited constants or variables which provide

the Stamp with information on how to execute the instruction.
Variable is sometimes needed to store a result.

This is of course a general format, not all statements follow it. Here's a few actual examples of PBASIC statements for the BASIC Stamp 1:

```
HIGH 4 - Set I/O pin 4 to logical 1 (5V)
LET b0 = $20 - Load the hexadecimal constant '20' into RAM location b0 (byte 0)
SEROUT 1, N2400, ("Hello World") - Transmit the 11 characters "HELLO WORLD" on I/O pin 1 at 2400 baud (inverted)
```

The important thing to notice is that the mnemonics and acronyms in PBASIC read like english. It's easy to write software for the BASIC Stamp because the language allows you to write your program like a series of events, or like a story.

It's critical for a microcontroller to be able make decisions, ("choose you own adventure" novels anyone?) and PBASIC has a few statements which allow this by conditionally jumping from one point in your program to another.

```
IF Variable = Variable/Constant THEN Line Label
```

This is a simplified IF THEN statement, (simplified meaning there's no ELSE) which allows the BASIC Stamp to test a condition and jump to another point in the program if that condition is true. For example:

```
IF IN1 = 1 THEN Pin1_Is_High
```

IN1 is a PBASIC reserved variable which represents the logic voltage state on I/O pin 1. If this is high (5V) then the BASIC Stamp will jump to the point in the program immediately following the label

Pin1_Is_High.

Of course there's much more to PBASIC than this, but this is only meant to touch the surface. For more information on PBASIC, refer to the BASIC Stamp Manual which can be downloaded in PDF format from [Parallax's website](http://www.parallax.com).

PIC Software:

Although PIC's can be programmed in a range of higher level languages like BASIC, C, Java and JAL, I prefer assembly language. Assembly language requires you to tell the microcontroller exactly what to do one instruction at a time, unlike PBASIC where you ask it to perform a task and it will execute any number of instructions.

Single instructions perform simple and somewhat useless operations

by themselves, and therefore must be grouped together to perform useful tasks. The code below performs the same task as the PBASIC statement `HIGH 4`:

```
BSF STATUS, RP0
BCF TRISB, 4
BCF STATUS, RP0
BSF PORTB, 4
```

BSF stands for **Bit Set File** register. BCF stands for **Bit Clear File** register. The first and third lines are required to give the CPU access to the required file registers TRISB and PORTB. By clearing bit 4 in TRISB I/O pin 4 of port B is put into output mode, (tristate mode is disabled) and by setting bit 4 in PORTB I/O pin 4 of port B is driven high. Confused? If you're having difficulty following, it may be a good idea to start with a BASIC Stamp. (I did)

PIC's have a "working" register (W register) which holds one operand for most instructions, allowing stored instructions to take up no more than 16bits. (depending on the PIC) The following instructions demonstrate this:

```
MOVLW Constant - Move Literal to Working register
MOVWF File Register - Move Working register to File register
ADDWF File Register - Add Working register to File register
INCF File Register - Increment File register
```

By using the MOVLW instruction the operand in W can be predefined. If the PIC had to remember two file registers or constants to use as operands the instruction size would too large, at least 22 bits.

Flow control (decision making) is not so straight forward with the PIC. A few instructions have variations which will skip the next instruction if a certain condition is met. Some of these are:

```
BTFSS File Register, Bit - Bit Test File register, Skip if Set
BTFSC File Register, Bit - Bit Test File register, Skip if Clear
INCFSZ File Register - Increment File register, Skip if Zero
DECFSZ File Register - Decrement File register, Skip if Zero
```

The first two instructions will skip the following instruction if the bit of the file register being tested is set, (or clear) and the next two instructions will increment or decrement a file register and skip the next instruction if the result is zero. (incrementing past 255 rolls over to zero) If the next instruction is an unconditional branch, (GOTO) then flow control has been achieved.

As you can see, writing software for the PICmicro is somewhat more involved. If you're just looking to get started with microcontrollers I'd

recommend you get yourself a BASIC Stamp, (even a BS1) and download the BASIC Stamp Manual from [Parallax](#). In time you will probably move on to PIC's or another assembly language micro.

Conclusion:

That's about as much as I can write in one night, I hope it is of use. What I've tried to do here is simply to give an insight to the world of microcontrollers for those who have little or no knowledge of them. If you choose to start working with a microcontroller there's always plenty of information and tutorials online, not to mention manufacturers documentation.

If you have any comments or questions please don't hesitate to [contact](#) me.

[Return To Top](#)

Last Updated: 18/02/2007

[Home Page](#)